

ANÁLISIS DEL RETO

Gabriel Andrés Moya Caravaca - g.moyac@uniandes.edu.co

Gerardo Rocha Linares - 202513388 - g.rochal2@uniandes.edu.co

Edgar Daniel Gonzalez Martinez - 202524741 - ed.gonzalezm1@uniandes.edu.co

Requerimiento 1

El Requerimiento 1 busca filtrar las ventas de un modelo específico dentro de un rango de precio base. Es decir. Por esa razón, el ejercicio se implementó usando una tabla de hash donde cada llave es un modelo de vehículo y cada valor es un árbol rojo-negro ordenado por precio base. En ese árbol, cada nodo tiene como llave un Base Price específico y como valor una lista de ventas con ese mismo precio. Así, primero se busca el modelo solicitado, luego se consulta en el árbol el rango de precios indicado y finalmente se recorren las ventas encontradas para calcular el total, el promedio de precio y ordenar los resultados según el criterio pedido.

Descripción

Breve descripción de como abordaron la implementación del requerimiento

Entrada	*-Modelo por analizar (por ejemplo: "GLC"). *Valor mínimo del rango de precio base (USD) a filtrar (por ejemplo: 45000). *Valor máximo del rango de precio base (USD) a filtrar (por ejemplo: 60000)
Salidas	El requerimiento debe mostrar el tiempo de ejecución, el total de ventas encontradas para el modelo y rango de precio consultado, el promedio del precio base y la lista de ventas filtradas con sus datos principales: modelo, año, tipo de combustible, color, precio base, horsepower y turbo. Los resultados deben ir ordenados por precio base ascendente; si hay empate, por horsepower descendente y luego por color alfabéticamente.
Implementado (Sí/No)	Si, fue implementado por Gabriel Andrés Moya Caravaca.

Análisis

La implementación del Requerimiento 1 es adecuada porque aprovecha la forma en que se cargaron los datos. Al tener una tabla hash organizada por modelo, el algoritmo puede llegar directamente al grupo de ventas del modelo consultado sin tener que recorrer todo.

Análisis de complejidad

```
def req_1(catalog, model, min_price, max_price):  
    """  
    Retorna el resultado del requerimiento 1  
    """  
    start_time = get_time()  
    model = model.upper()  
    model_tree = mp.get(catalog["model_price"], model)  
    filtered_sales = al.new_list()  
    total_price = 0  
  
    if model_tree is not None:  
        price_groups = rbt.values(model_tree, min_price, max_price)  
        current_group = price_groups["first"]  
  
        while current_group is not None:  
            sales_list = current_group["info"]  
  
            for i in range(al.size(sales_list)):  
                sale = al.get_element(sales_list, i)  
                al.add_last(filtered_sales, sale)  
                total_price += sale["base_price"]  
  
            current_group = current_group["next"]  
  
        al.merge_sort(filtered_sales, compare_req_1)  
  
        total = al.size(filtered_sales)  
        avg_price = 0  
        if total > 0:  
            avg_price = total_price / total  
  
        shown_sales = get_first_last(filtered_sales, 6)  
  
        end_time = get_time()  
        delta = delta_time(start_time, end_time)  
  
    return delta, total, avg_price, shown_sales
```

La complejidad del Requerimiento 1 depende principalmente de la cantidad de ventas que cumplen el filtro. La búsqueda del combustible en la tabla hash tiene complejidad promedio $O(1)$. Luego, la consulta del rango en el árbol rbt tiene complejidad $O(\log h)$, donde h es la cantidad de potencias distintas registradas en el árbol del combustible consultado. Después se recorren las r ventas que cumplen el filtro, lo cual cuesta $O(r)$, pero la complejidad sigue siendo dominada por el siguiente. Finalmente esas ventas se ordenan con merge sort, lo que cuesta $O(r \log r)$. Por eso, la complejidad total del requerimiento es $O(\log h + r \log r)$. Si el rango incluyera todas las ventas la complejidad sería $O(n \log n)$.

Requerimiento 2

Descripción

```
def req_2(catalog, combustible, hp_min, hp_max):  
    start_time = get_time()  
    hp_min = int(hp_min)  
    hp_max = int(hp_max)  
    combustible = combustible.upper()  
    arbol = mp.get(catalog["fuel_hp"], combustible) #saco el arbol que  
    vehiculos_objetivo = al.new_list()  
    precio_total = 0  
    hp_total = 0  
  
    if arbol is not None:  
        #me interesa revisar en un rango de (hp) dado  
        vehiculos_rango_hp = rbt.values(arbol, hp_min, hp_max) #saco los  
        vehiculos_por_potencia = vehiculos_rango_hp["first"] #saco el  
  
        while vehiculos_por_potencia is not None:  
            lista_de_ventas = vehiculos_por_potencia["info"]  
            for i in range(al.size(lista_de_ventas)):  
                vehiculo = al.get_element(lista_de_ventas, i)  
                al.add_last(vehiculos_objetivo, vehiculo)  
                precio_total += vehiculo["base_price"]  
                hp_total += vehiculo["horsepower"]  
  
            vehiculos_por_potencia = vehiculos_por_potencia["next"]  
  
        al.merge_sort(vehiculos_objetivo, compare_req_2)  
        total = al.size(vehiculos_objetivo)  
        precio_promedio = 0  
        hp_promedio = 0  
        if total > 0:  
            precio_promedio = precio_total / total  
            hp_promedio = hp_total / total  
  
        shown_sales = get_first_last(vehiculos_objetivo, 6)  
  
        end_time = get_time()  
        delta = delta_time(start_time, end_time)  
  
    return delta, total, precio_promedio, hp_promedio, shown_sales
```

El Requerimiento 2 busca filtrar las ventas por tipo de combustible y por un rango de horsepower, para identificar los vehículos vendidos que cumplen esas condiciones y poder analizar su potencia según el tipo de motorización. Por esa razón, el ejercicio se implementó de la siguiente forma, se usó una tabla de hash donde cada llave es un tipo de combustible y cada valor es un árbol rbt ordenado por horsepower. Así, primero se busca el combustible solicitado, luego se consulta en el árbol el rango de potencia indicado y finalmente se recorren las ventas encontradas para calcular los promedios y ordenar los resultados.

Entrada	*Tipo de combustible a analizar (por ejemplo: "Electric"). *Valor mínimo de horsepower a filtrar (por ejemplo: 200). *Valor máximo de horsepower a filtrar (por ejemplo: 450).
---------	--

Salidas	El requerimiento debe mostrar el tiempo de ejecución, el total de ventas encontradas, el promedio de precio, el promedio de horsepower y la lista de ventas filtradas con sus datos principales, ordenadas por horsepower, precio base y modelo.
Implementado (Sí/No)	Sí, fue implementado por Gerardo Rocha Linares.

Análisis de complejidad

La complejidad del Requerimiento 2 depende principalmente de la cantidad de ventas que cumplen el filtro. La búsqueda del combustible en la tabla hash tiene complejidad promedio $O(1)$. Luego, la consulta del rango en el árbol rbt tiene complejidad $O(\log h)$, donde h es la cantidad de potencias distintas registradas en el árbol del combustible consultado. Después se recorren las r ventas que cumplen el filtro, lo cual cuesta $O(r)$, pero la complejidad sigue siendo dominada por el siguiente. Finalmente esas ventas se ordenan con merge sort, lo que cuesta $O(r \log r)$. Por eso, la complejidad total del requerimiento es $O(\log h + r \log r)$. Si el rango incluyera todas las ventas la complejidad sería $O(n \log n)$.

Análisis

La implementación del Requerimiento 2 es eficiente porque aprovecha la forma en que fueron cargados los datos. Al tener una tabla hash organizada por tipo de combustible, el algoritmo puede llegar directamente al grupo de ventas que interesa, sin revisar todos los registros.

Requerimiento 3

Descripción

```
def req_3(catalog, year, fuel_type, min_price, max_price):
    """
    Retorna el resultado del requerimiento 3
    """
    # TODO: Modificar el requerimiento 3
    tiempo_inicio = get_time()
    lista_filtrada = al.new_list()
    precio_total = 0
    tipo_combustible = fuel_type.upper().strip()

    años = mp.get(catalog["year_fuel_price"], year)
    if años is not None:
        combustible_arbol = mp.get(años, tipo_combustible)
        if combustible_arbol is not None:
            precio = rbt.values(combustible_arbol, min_price, max_price)
            actual = precio["first"]
            while actual is not None:
                for i in range(al.size(actual["info"])):
                    venta = al.get_element(actual["info"], i)
                    al.add_last(lista_filtrada, venta)
                    precio_total += venta["base_price"]
                    actual = actual["next"]

            al.merge_sort(lista_filtrada, comparacion_req_3)
            total = al.size(lista_filtrada)
            promedio_precio = precio_total / total if total > 0 else 0

    tiempo_final = get_time()
    tiempo = tiempo_final - tiempo_inicio

    return tiempo, total, promedio_precio, lista_filtrada
```

El Requerimiento 3 busca filtrar las ventas de un año y un tipo de combustible específico dentro de un rango de precio base. O sea, el usuario ingresa el año, el combustible, un precio mínimo y un precio máximo, y el sistema debe encontrar las ventas que cumplan esas tres condiciones. Por esta razón el requerimiento se implementó usando una tabla hash organizada primero por año. Dentro de cada año hay otra tabla hash organizada por tipo de combustible, y cada combustible apunta a un árbol rojo-negro ordenado por precio base. Así, primero se busca el año solicitado, luego el tipo de combustible dentro de ese año y finalmente se consulta en el árbol el rango de precios indicado. Después se recorren las ventas encontradas, se calcula el total, el promedio de precio y se ordenan los resultados según el

criterio pedido.

Entrada	*Año (por ejemplo: 2023). *Tipo de combustible (por ejemplo: "Hybrid").
---------	--

	*Valor mínimo del rango de precio base (USD) (por ejemplo: 60000). *Valor máximo del rango de precio base (USD) (por ejemplo: 120000)
Salidas	El requerimiento debe mostrar el tiempo de ejecución, el total de ventas encontradas, el promedio del precio base y la lista de ventas filtradas con sus datos principales, ordenadas por precio base descendente, horsepower descendente y Turbo.
Implementado (Sí/No)	Si, fue implementado por Edgar Daniel Gonzalez Martinez.

Análisis de complejidad

La complejidad del Requerimiento 3 depende principalmente de r , que es la cantidad de ventas que cumplen el filtro, y de h , que es la cantidad de precios distintos en el árbol rojo-negro del año y combustible consultados. Primero se busca el año y el combustible en tablas hash, lo cual cuesta $O(1)$ en promedio. Luego se consulta el rango de precios en el árbol rojo-negro, con costo $O(\log h)$. Después se recorren las r ventas encontradas, lo cual cuesta $O(r)$, pero este costo queda dominado por el siguiente. Finalmente, esas ventas se ordenan con merge sort, lo que cuesta $O(r \log r)$. Por eso, la complejidad total queda como **$O(\log h + r \log r)$** . En el peor caso, si el rango incluye casi todas las ventas del año y combustible consultados, r se acerca a n , y la complejidad queda cercana a **$O(n \log n)$** .

Análisis

El Requerimiento 3 es eficiente y cuenta con una estructura optima porque usa una tabla hash por año y otra por tipo de combustible, lo que permite llegar al grupo de ventas necesario en $O(1)$ promedio. Luego, el árbol rojo-negro consulta el rango de precios en $O(\log h)$, donde h es la cantidad de precios distintos. Finalmente, se ordenan las r ventas filtradas con merge sort en $O(r \log r)$. Por eso, el algoritmo trabaja principalmente sobre los resultados encontrados y no sobre todo.

Requerimiento 4

Descripción

```
def req_4(catalog, year, n):
    """
    Retorna el resultado del requerimiento 4
    """
    start_time = get_time()

    year_map = mp.get(catalog["year_model_stats"], year)
    top_models = al.new_list()
    total_models = 0

    if year_map is not None:
        total_models = mp.size(year_map)
        stats_values = mp.value_set(year_map)
        ranking = pq.new_heap(is_min_heap=False)

        for i in range(al.size(stats_values)):
            stats = al.get_element(stats_values, i)
            result = build_req_4_result(stats)
            priority = (result["sales_count"], result["avg_price"])
            pq.insert(ranking, priority, result)

        limit = n
        if total_models < limit:
            limit = total_models

        for i in range(limit):
            al.add_last(top_models, pq.remove(ranking))

    al.merge_sort(top_models, compare_req_4)

    end_time = get_time()
    delta = delta_time(start_time, end_time)

    return delta, total_models, top_models
```

El Requerimiento 4 busca identificar, para un año específico, los N modelos con mayor cantidad de vehículos vendidos. Además del ranking por ventas, para cada modelo se deben calcular datos como el precio promedio, el horsepower promedio, el porcentaje de ventas con Turbo y la venta con mayor horsepower dentro de ese año. Por esa razón el ejercicio se implementó usando una tabla hash organizada por año. Cada llave corresponde a un año y su valor es otra tabla hash con las estadísticas acumuladas por modelo. Para resolver la consulta, primero se busca el año solicitado, luego se recorren las estadísticas de cada modelo vendido en ese año y se insertan en una cola de prioridad. Esta cola permite priorizar los modelos con mayor número de ventas y, en caso de empate, con mayor precio promedio. Finalmente, se extraen los primeros N modelos y se ordenan.

Entrada	*Año (por ejemplo: 2021). *Cantidad N de modelos con mayor número de vehículos vendidos (por ejemplo: N = 5)
Salidas	El requerimiento debe mostrar el tiempo de ejecución, el total de modelos considerados para el año consultado y el ranking de los N modelos con más ventas. Para cada modelo se debe presentar su número de vehículos vendidos, precio promedio, horsepower promedio, porcentaje de Turbo = Yes y la información de la venta con mayor horsepower.
Implementado	Si, fue implementado por G07.

Análisis de complejidad

La complejidad del Requerimiento 4 depende sobre todo de m, que es la cantidad de modelos distintos vendidos en el año consultado, y de N, que es la cantidad de modelos que se quieren mostrar en el ranking. Primero se busca el año en la tabla hash, lo cual en promedio cuesta $O(1)$. Luego se recorren los m modelos de ese año y cada uno se mete en una cola de prioridad, lo que cuesta $O(m \log m)$. Después se sacan los N mejores modelos del heap, con costo $O(N \log m)$, y se ordenan con merge sort en $O(N \log N)$. Como N no puede ser mayor que m, esos pasos no cambian mucho el costo principal del algoritmo. Por eso, la complejidad total queda como $O(m \log m)$.

Análisis

La implementación del Requerimiento 4 es optima ya que porque aprovecha que los datos ya están agrupados por año y por modelo. Entonces, en vez de recorrer todas las ventas cada vez que se hace la consulta, primero se busca directamente el año en una tabla hash. Si el año existe, se obtiene el mapa de modelos vendidos en ese año y se trabaja solo con esos modelos.

Requerimiento 5

Descripción

```
def req_5(catalog, horsepower, delta, n):
    mapa_colores = mp.new_map(20)
    total_vehiculos = 0

    if arbol is not None:
        vehiculos_rango_hp = rbt.values(arbol, hp_min, hp_max) # saco la lista de vehiculos en el rango

        if vehiculos_rango_hp is not None:
            vehiculos_por_potencia = vehiculos_rango_hp["first"]

            while vehiculos_por_potencia is not None:
                lista_de_ventas = vehiculos_por_potencia["info"]

                for i in range(al.size(lista_de_ventas)):
                    vehiculo = al.get_element(lista_de_ventas, i)

                    color = vehiculo["color"]
                    hp = vehiculo["horsepower"]

                    total_vehiculos += 1

                    info_color = mp.get(mapa_colores, color)

                    if info_color is None:
                        info_color = {
                            "color": color,
                            "total_ventas": 1,
                            "hp_total": hp,
                            "hp_promedio": 0
                        }

                        mp.put(mapa_colores, color, info_color)

                    else:
                        info_color["total_ventas"] += 1
                        info_color["hp_total"] += hp
                        mp.put(mapa_colores, color, info_color)

                vehiculos_por_potencia = vehiculos_por_potencia["next"]

    lista_colores = al.new_list()

    colores = mp.key_set(mapa_colores)

    for i in range(al.size(colores)):
        color = al.get_element(colores, i)
        info_color = mp.get(mapa_colores, color)

        info_color["hp_promedio"] = (info_color["hp_total"] / info_color["total_ventas"])

        al.add_last(lista_colores, info_color)

    al.merge_sort(lista_colores, compare_req_5)

    top_colores = al.new_list()

    limite = n
    if al.size(lista_colores) < n:
        limite = al.size(lista_colores)

    for i in range(limite):
        color_info = al.get_element(lista_colores, i)
        al.add_last(top_colores, color_info)

    end_time = get_time()
    delta_time_req = delta_time(start_time, end_time)

    return delta_time_req, total_vehiculos, top_colores
```

El Requerimiento 5 busca encontrar los colores más vendidos dentro de un rango de horsepower. Con la potencia y el delta se arma un rango, y luego se revisan las ventas que tengan un horsepower dentro de ese intervalo. Después, se cuentan las ventas por color para sacar el Top N de colores más vendidos. Por esa razón, el ejercicio se implementó usando un árbol rojo-negro ordenado por horsepower. Cada nodo del árbol tiene como llave una potencia específica y como valor una lista de ventas con esa misma potencia. Primero se calcula el rango de horsepower usando el valor de referencia y el delta. Luego se consulta en el árbol rojo-negro qué ventas están dentro de ese rango. Después, esas ventas se agrupan por color en una tabla hash, donde se guarda cuántas ventas tiene cada color y la suma de horsepower para luego calcular el promedio. Finalmente, los colores se ordenan por cantidad de ventas descendente, luego por horsepower promedio descendente y, si sigue el empate, por color alfabéticamente.

Entrada	*Horsepower de referencia (por ejemplo 345). *Delta de Horsepower: Valor de incremento y decremento aplicado al horsepower de referencia para definir el rango de interés. *Cantidad N de colores a mostrar.
Salidas	El requerimiento debe mostrar el tiempo de ejecución, el total de vehículos que cumplen el rango de horsepower y el Top N de colores más vendidos dentro de ese rango. Para cada color se debe presentar el nombre del color, el número total de vehículos vendidos y el horsepower promedio de esos vehículos
Implementado (Sí/No)	Si, fue implementado por G07.

Análisis de complejidad

La complejidad del Requerimiento 5 depende de r , la cantidad de vehículos en el rango de horsepower, y de c , la cantidad de colores distintos encontrados. La consulta del rango en el árbol rojo-negro cuesta $O(\log h)$, donde h es la cantidad de potencias distintas, pero la complejidad en realidad se la da el siguiente término. Luego se recorren los r vehículos y se agrupan por color con una tabla hash en $O(r)$. Finalmente, los c colores se ordenan con merge sort en $O(c \log c)$. La complejidad total es $O(r + c \log c)$. En el peor

caso, si el rango incluye casi todos los vehículos y cada uno tiene un color distinto, la complejidad se aproxima a $O(n \log n)$. En la práctica, como c es pequeño y acotado, la complejidad real se acerca a $O(n)$.

Análisis

La implementación es optima y eficiente porque no empieza recorriendo todas las ventas. Como el requerimiento filtra por un rango de horsepower, se usa directamente el árbol rojo-negro `catalog["by_horsepower"]`, que ya está organizado por potencia. Esto permite llegar al rango solicitado de forma más rápida y trabajar solo con las ventas que realmente pueden servir para la consulta.

Requerimiento 6

Descripción

El requerimiento 6 busca determinar según un rango de años y de precios para cierta cantidad M , por esto y implementamos un árbol rojo - negro por año, esto para filtrar por rango de años más fácil y más rápido, para después dentro de esas ventas voltear por rango de precio. Las ventas que cumplen ambas cosas se agrupan por modelo en una tabla de hash. Para cada modelo se calcula el promedio, desviación estándar y la estabilidad del precio, pero los modelos con promedio de 0 se excluyen. En lo último se utiliza una cola de prioridad mínima para sacar los M modelos más estables y se retornan ordenados ascendentemente por estabilidad.

Entrada	*Rango de años (por ejemplo 2020, 2025) *Rango de precio base en dólares (40000, 120000) *Cantidad de modelos a mostrar (por ejemplo = 5)
Salidas	Salidas el tiempo de ejecución, número total de modelos y para cada modelo: número de ventas, promedio de precio, desviación estándar, estabilidad, horsepower, promedio e información de la venta más representativa
Implementado (Sí/No)	Si, fue implementado por G07.

Análisis de complejidad

La complejidad, el requerimiento seis depende de la cantidad de ventas que cumplen el filtro y de M además de la cantidad de modelos distintos encontrados, por lo que primero se consulta el rango de años en el albor rojo negro con costo $O(\log n)$ dónde N es la cantidad de años distintos, luego se recorren las ventas filtradas por precio y se agrupan por modelo en una tabla de Hash, con costo de $O(N)$ después para acá modelo, se calculan la desviación, el promedio y la estabilidad, además de la venta representativa, recorriendo sus ventas, lo cual también cuesta $O(N)$. Cada

```

def req_s(catalog, min_year, max_year, min_price, max_price, cantidad_s):
    """
    Retorna el resultado del requerimiento s
    """
    # TODO: Modificar el requerimiento s
    tiempo_inicial = get_time()
    lista_filtrada = al.new_list()
    total_modelos = 0

    años_filtrados = set.keys(catalog["by_year"], min_year, max_year)
    año_actual = años_filtrados["first"]
    while año_actual is not None:
        lista_ventas = set.get(catalog["by_year"], año_actual["info"])
        for i in range(al.size(lista_ventas)):
            venta = al.get_element(lista_ventas, i)
            if min_price <= venta["base_price"] <= max_price:
                al.add_back(lista_filtrada, venta)
        año_actual = años_filtrados["next"]

    ventas_por_modelo = sp.new_map(sp.size(catalog["model_price"]))
    for i in range(al.size(lista_filtrada)):
        venta = al.get_element(lista_filtrada, i)
        if sp.get(ventas_por_modelo, venta["model"]) is None:
            sp.put(ventas_por_modelo, venta["model"], al.new_list())
        al.add_back(sp.get(ventas_por_modelo, venta["model"]), venta)

    ranking = pq.new_heap()
    lista_modelos = sp.key_val(ventas_por_modelo)

    for i in range(al.size(lista_modelos)):
        modelo = al.get_element(lista_modelos, i)
        lista_ventas = sp.get(ventas_por_modelo, modelo)
        cantidad_ventas = al.size(lista_ventas)
        total_modelos += 1

        suma_precios = 0
        suma_bp = 0
        for j in range(cantidad_ventas):
            venta = al.get_element(lista_ventas, j)
            suma_precios += venta["base_price"]
            suma_bp += venta["base_price"]

        su = suma_precios / cantidad_ventas
        promedio_bp = suma_bp / cantidad_ventas

        if su != 0:
            suma_desviacion = 0
            for j in range(cantidad_ventas):
                venta = al.get_element(lista_ventas, j)
                suma_desviacion += (venta["base_price"] - su) ** 2
            sigma = (suma_desviacion / cantidad_ventas) ** 0.5
            estabilidad = sigma / su

            venta_representativa = None
            diferencia_minima = float("inf")
            for j in range(cantidad_ventas):
                venta = al.get_element(lista_ventas, j)
                diferencia = abs(venta["base_price"] - su)
                if diferencia < diferencia_minima:
                    diferencia_minima = diferencia
                    venta_representativa = venta
            elif diferencia == diferencia_minima:
                if venta["base_price"] < venta_representativa["base_price"]:
                    venta_representativa = venta
                elif venta["base_price"] == venta_representativa["base_price"]:
                    if venta["year"] < venta_representativa["year"]:
                        venta_representativa = venta

            pq.insert(ranking, (estabilidad, sigma, modelo), {
                "modelo": modelo,
                "cantidad_ventas": cantidad_ventas,
                "su": su,
                "sigma": sigma,
                "estabilidad": estabilidad,
                "promedio_bp": promedio_bp,
                "venta_representativa": venta_representativa
            })
        else:
            total_modelos += 1

    lista_top = al.new_list()
    for i in range(min(cantidad_s, total_modelos)):
        al.add_back(lista_top, pq.remove(ranking))

    tiempo_final = get_time()
    tiempo = tiempo_final - tiempo_inicial
    return tiempo, total_modelos, lista_top

```

modelo se inserta en la cola de prioridad con costo $O(N \log n)$. Por esto, la complejidad queda como $O(n + n \log n)$.

